

Oefententamen Inleiding Programmeren voor Bèta-Gamma

Naam:

.....

- Je zult dit tentamen maken met behulp van de online editor. Om toegang te krijgen:
 - Ga naar **progbg.proglab.nl/exams**.
 - Voer de tentamencode in: .
 - Wacht tot de docent je collegekaart of ID heeft gecontroleerd.
 - De surveillant zal jouw persoonlijke **PIN** invoeren.
 - Terwijl je wacht, kun je doorgaan met het lezen van deze tentameninstructies.
- Dit digitale tentamen bestaat uit 6 opgaven waarin je korte Python-programma's zult schrijven.
- Je hebt minstens 3 correcte oplossingen nodig voor een voldoende.
- Gebruik één bestand genaamd `ipbg_exam0.py` voor alle oplossingen. Dit bestand is al geopend in de online exameditor.
- Je mag de cursuswebsite als bron gebruiken (open `progbg.proglab.nl` in een nieuwe tab). Houd er rekening mee dat:
 - je geen enkele andere website kunt gebruiken,
 - je niet kan geen gebruik maken van bestaande code die je voor dit tentamen hebt geschreven,
 - je geen hulp kunt krijgen bij programmeren tijdens het tentamen.
- Je wordt **uitsluitend beoordeeld op de correctheid** van je oplossingen. Codeontwerp, commentaar en stijl zijn niet van belang, dus je hoeft je daar geen zorgen over te maken.
- De editor heeft een Checkpy-knop waarmee je tijdens het tentamen je oplossingen kunt controleren voor persoonlijk gebruik. Maar, de beoordeling gebeurt handmatig wordt uitgevoerd. Dus zelfs als je oplossingen niet slagen voor Checkpy, kunnen ze nog steeds worden geaccepteerd tijdens het beoordelingsproces.
- Gebruik geen externe modules zoals `numpy`, `csv` of andere, tenzij de opgave specifiek aangeeft dat je dat mag.
- **Inleveren van het tentamen.** Wanneer je alle opgaven hebt afgerond:
 - gebruik de **indienen-knop** op de website om ervoor te zorgen dat je laatste wijzigingen worden opgeslagen,
 - sluit de browser.
 - **ga naar de docent** met je **collegekaart of ID** en dit tentamenblad,
 - laat de docent je werk als voltooid markeren,
 - lever dit tentamenblad (met je naam erop) in.

Opdracht 1: Fuz

Schrijf een functie genaamd `fuz(n)`. Deze functie retourneert een lijst met alle *fuz*-getallen tussen 1 en `n` (inclusief 1, exclusief `n` zelf). We definiëren een *fuz*-getal als een getal dat deelbaar is door 3 of door 5, maar niet door beide.

Voorbeeld

```
fuz_list = fuz(35)
print(fuz_list)
```

Verwachte Output

```
[3, 5, 6, 9, 10, 12, 18, 20, 21, 24, 25, 27, 33]
```

Tip: je kunt de `%`-operator gebruiken om te testen of een getal deelbaar is.

Opdracht 2: Woorden tellen

Schrijf een functie `count_elements(text)` die telt hoeveel elementen iemand opsomt die beginnen met een klinker (a, e, i, o, u). Deze functie heeft de string `text` als invoer, waarin elementen van een bepaalde categorie worden opgesomd, gescheiden door komma's. De uitvoer van de functie moet het aantal elementen zijn dat in de tekst wordt vermeld.

Zorg ervoor dat je functie ook werkt als een element begint met een teken dat niet in het alfabet staat en met zowel hoofdletters als kleine letters.

Voorbeeld

```
example_text = "Bananas, oranges, apples, strawberrys, grapes."
print(count_elements(example_text))
```

Verwachte Output

```
2
```

Opdracht 3: GCD

De grootste gemene deler (*greatest common divisor*: gcd) van twee gehele getallen is het grootste positieve gehele getal dat beide gegeven getallen deelt zonder dat er een rest overblijft.

Als voorbeeld nemen we de getallen 12 en 18. De delers van 12 zijn 1, 2, 3, 4, 6 en 12. De delers van 18 zijn 1, 2, 3, 6, 9 en 18. Het grootste getal dat in beide lijsten voorkomt en zowel 12 als 18 deelt, is 6. Dus de grootste gemene deler van 12 en 18 is 6.

Write a function `gcd(number1, number2)`, that takes two integers as input and determines the greatest common divisor.

Voorbeeld

```
print(gcd(10, 6))  
print(gcd(6, 12))  
print(gcd(1000, 240))
```

Verwachte Output

```
2  
6  
40
```

Opdracht 4: Auto-Reformat

Schrijf een functie genaamd `auto_reformat(text)`, die een slecht opgemaakte tekst aanneemt en de opmaak verbetert volgens de volgende richtlijnen:

- Elke zin begint met een hoofdletter.
- Er staat slechts één spatie tussen elke zin en tussen elk woord.
- Alle woorden worden op de juiste manier geschreven:
 - Een woord *mag* beginnen met een hoofdletter (ook als het niet het eerste woord van de zin is).
 - De rest van het woord moet in kleine letters staan.

Voorbeeld

```
text = "hello,   world. This tExt NEeds   some cleaning   up.   "  
result = auto_reformat(text)  
print(result)
```

Verwachte Output

```
Hello, world. This text Needs some cleaning up.
```

Tip: kijk goed naar bestaande string-methodes die het je aanzienlijk makkelijker kunnen maken, zoals `.isupper()`, `.lower()`, `.upper()`, `.capitalize()`, `.strip()` en `.split()`.

Opdracht 5: Magiërsvierkant

Schrijf een functie `magic_square(square)` die controleert of een gegeven `square` matrix (lijst van lijsten) een **magiërsvierkant** is. Een magiërsvierkant is een vierkante matrix van getallen waarbij de som van elke rij en de som van elke kolom hetzelfde is. (Normaal gesproken moeten de diagonalen ook dezelfde som hebben, maar dat negeren we voor deze opdracht.)

Je functie moet een 2D-lijst (lijst van lijsten) ontvangen die de vierkante matrix voorstelt en `True` retourneren als de matrix een magiërsvierkant is en `False` als dat niet zo is. Bekijk de voorbeelden hieronder voor een beter begrip van de opdracht. Denk eraan, de code moet werken voor matrices van elke omvang van ten minste 1x1. Je mag ervan uitgaan dat de invoer altijd een lijst van lijsten is. Je mag ervan uitgaan dat de invoermatrix altijd vierkant is (d.w.z. het aantal rijen is gelijk aan het aantal kolommen).

Voorbeeld 1

```
square1 = [  
    [16, 2, 3, 13],  
    [5, 11, 10, 8],  
    [9, 7, 6, 12],  
    [4, 14, 15, 1]  
]  
print(magic_square(square1)) # True (alle rijen en kolommen tellen op tot 34).
```

Verwachte Output

```
True
```

Voorbeeld 2

```
square2 = [  
    [16, 3, 2, 13],  
    [5, 11, 6, 8],  
    [9, 7, 10, 12],  
    [4, 14, 15, 1]  
]  
print(magic_square(square2))  
# False (niet alle rijen en kolommen tellen op tot hetzelfde).  
# (Bijvoorbeeld, de som van rij 0 is 34, maar die van rij 1 is 30.)
```

Verwachte Output

```
False
```

Opdracht 6: Copriem

Schrijf een functie `coprimes(n)` die alle combinaties van getallen tussen 2 (inclusief) en n (exclusief) vindt die copriem zijn maar zelf niet een priemgetal zijn. Twee getallen zijn copriem als de grootste gemeenschappelijke deler die ze hebben 1 is. Bijvoorbeeld 4 en 9 zijn twee getallen die zelf niet een priemgetal zijn, maar wel gezamenlijk copriem. De output met een lijst zijn van tuples van alle paren die aan deze voorwaarden voldoen. Alle paren in de lijst moeten uniek zijn.

Voorbeeld 1

```
print(coprimes(10))
```

Verwachte Output

```
[(4, 9), (8, 9)]
```

Voorbeeld 2

```
print(coprimes(20))
```

Verwachte Output

```
[(4, 9), (4, 15), (8, 9), (8, 15), (9, 10), (9, 14), (9, 16), (14, 15), (15, 16)]
```